

Threat Model

What Shadow protects, how it protects it, and — with equal prominence — what it does not. Every claim maps to a mechanism in the source. If our marketing ever disagrees with this document, this document wins.

Applies to **Shadow v2.6.x** Status **living document** Verify your posture `shadow doctor --privacy`

The model is untrusted input. Everything a model emits — text, tool calls, “reasoning” — is treated the way a browser treats a web page: potentially adversarial output to be **contained**, not trusted. Containment is four mechanisms working together: the **workspace jail** (file tools), the **OS sandbox** (shell), the **approval gate** (autonomy), and **scoped secrets** (env allowlist + encrypted vault). Nothing in Shadow’s safety story depends on the model being well-behaved.

1 Trust model

THING	TRUST LEVEL	WHY
The model’s output	Untrusted	May be wrong, manipulated, or adversarial (prompt injection)
Fetched web content / search results	Untrusted data	Never instructions — see limit №2 overleaf
A cloned repo’s <code>shadow.config.json</code>	Untrusted	De-fanged on load: no key redirect, no autonomy grant, no startup exec
Your global <code>~/.shadow/</code> config	Trusted by design	Your file on your machine — an attacker there already owns the account
Configured MCP servers	Trusted once added	Run as your user, unsandboxed — vet before adding
The provider endpoint you configure	Trusted with your data	The conversation goes there; that is the product working
Local model runtime (llama.cpp / Ollama)	Trusted infrastructure	A trojaned runtime can open its own sockets
Env vars · CLI flags · the installer	Trusted	Standard local-machine trust; installs are signature-verified

Assets protected: your API keys · files outside the workspace · the workspace (from catastrophic commands) · your network posture (SSRF) · session privacy (zero telemetry).

2 Mechanisms & their real guarantees

2.1 Workspace jail — file tools

Every file-tool path resolves inside the workspace root (+ explicitly granted dirs). Symlinks collapse against the deepest existing ancestor, so a link pointing out — or a *not-yet-created* path — cannot escape.

LIMIT File tools only. Shell containment is the sandbox's job, below.

2.2 OS sandbox — `run_shell`

macOS (seatbelt): writes denied outside workspace/temp; `~/.shadow` unreadable and unwritable. **Linux** (bubblewrap): filesystem read-only, workspace + `/tmp` writable, `~/.shadow` masked by tmpfs, PID namespace unshared (blocks `/proc/*/environ` key theft). **Windows: no sandbox exists.**

LIMIT Fails open when the OS tool is missing (status is reported, never faked). Reads outside the workspace are allowed by design — pair with `sandboxNetwork: false` or `--offline` to cut read-then-exfiltrate. Network from the sandbox is allowed by default.

2.3 Secrets — scoped env + encrypted vault

Shell commands get an ~8-variable env allowlist; API keys are never in a subprocess environment. At rest, keys live in the vault: `script (N=216) → AES-256-GCM`, authenticated — wrong password or tampered file fails closed. The OS keychain caches the *derived* key, never the password.

LIMIT Bounded by master-password strength; an unlocked OS session can use the cached key; `SHADOW_VAULT_PASSWORD` is plaintext env for headless use.

2.4 Network guard — web tools

Refuses non-HTTP schemes and hosts resolving to loopback / private / link-local / CGNAT / cloud-metadata / v6-transition ranges. Connections are pinned to the validated IP (defeats DNS rebinding); every redirect hop is re-validated and re-pinned.

LIMIT Any *public* host is reachable once a network approval is granted — the approval gate, not the guard, controls that.

2.5 Denylist — a fat-finger guard, not a boundary

Catastrophic patterns (`rm -rf /`, `mkfs`, `dd of=/dev/...`, fork bombs) are refused even at full autonomy.

LIMIT Trivially bypassed by indirection (env vars, subshells, base64) — the source says so and so do we. It stops accidents; the sandbox stops attackers.

2.6 Approval gate — autonomy levels

`manual` → everything approved · `auto-read` → reads free · `auto-edit` (default) → edits free, exec approved · `full` → all auto-approved except denylist hits. One seam (the ApprovalGate) sees every gated call.

LIMIT At `full`, containment shrinks to sandbox + jail + denylist — a deliberate, user-chosen trade.

2.7 Untrusted project config — drive-by defense

A cloned repo's config cannot redirect keys, re-add secrets, raise autonomy, weaken the denylist/sandbox, swap the system prompt, widen the jail, run startup hooks, or register MCP servers. Project model presets are stripped of `gguf/ggufServer/ggufArgs` — a preset that spawns a binary pre-LLM would be zero-interaction RCE.

2.8 MCP — always execution-risk

MCP tools always gate as `exec`. A server's self-declared `readOnlyHint` is deliberately not trusted — a malicious server could label `delete_files` read-only.

2.9 Offline mode — subtractive by construction

`--offline` requires a local model or aborts; web tools are *not registered*; MCP is skipped; sandbox egress is denied; even the opt-in update check is suppressed. Mid-session switches to cloud models are refused.

LIMIT Shell-egress denial rides on the OS sandbox — on Windows, `--offline` cannot stop a shell command from reaching the network.

2.10 Zero telemetry — verifiable

No analytics, crash reporting, or phone-home. Egress = your provider + explicitly invoked web tools + (opt-in, off by default, once daily, payload-free) the update check. `shadow doctor --privacy` prints every egress path for your config — live or inactive — making no network calls itself. A source-level test pins the absence of identifiers.

3 What Shadow does not protect against

Stated with the same weight as the guarantees. If these matter to your environment, compensate deliberately.

- 01 **Malicious or steered model output is contained, never prevented.** The gate and sandbox limit what it can do — not what it tries.
 - 02 **Prompt injection has no mechanical fix.** “Fetched content is data” is framing plus the approval gate; a model that ignores the framing follows injected instructions up to the containment boundary. `full` autonomy + web tools + a sensitive workspace is a risk combination you choose.
 - 03 **The sandbox fails open** where `sandbox-exec` / `bwrap` is missing — and does not exist on Windows.
 - 04 **Read-then-exfiltrate is possible by default:** sandboxed commands can read most of the home directory and reach the network. Cut one side (`sandboxNetwork: false` or `--offline`) if this is in your model.
 - 05 **Redaction is best-effort.** Known key shapes and registered secrets are masked in logs; novel formats can leak. Redaction is never the reason something is safe to log.
 - 06 **A weak master password bounds the vault.** `script` slows brute force; it does not defeat a dictionary password against a stolen `vault.enc`.
 - 07 **MCP servers are your trust decision.** Once configured they run as your user, unsandboxed; the HTTP MCP transport is not SSRF-guarded.
 - 08 **The provider sees everything you send it.** That is the product working. Local models relocate this trust to your hardware and runtime — which you likewise trust.
 - 09 **Your own `~/ .shadow` is trusted.** Home-directory write access = owning the agent, and everything else you run.
 - 10 `--yolo` **voids the warranty.** Jail, sandbox, denylist, approvals — all off, by explicit flag.
 - 11 **Supply chain is out of runtime scope.** Release signatures are verified at install (ECDSA-P256, offline key, pinned in the installer); beyond that, the binary and dependencies are trusted at build time.
 - 12 **Side channels, kernel exploits, and a compromised OS** are out of scope entirely.
-

4 Platform matrix

CAPABILITY	MACOS	LINUX	WINDOWS
------------	-------	-------	---------

OS sandbox for <code>run_shell</code>	✓ seatbelt	✓ bubblewrap	✗ none
<code>~/ .shadow</code> invisible to shell	✓	✓ tmpfs mask	✗
Denylist patterns	✓ unix	✓ unix	🕒 add PowerShell rules via <code>denylistExtra</code>
Keychain for the vault key	✓ Keychain	✓ libsecret	🕒 DPAPI (any process as your user)
Workspace jail (file tools)	✓	✓	✓
Offline mode	✓ full	✓ full	🕒 shell egress not blockable

Windows is not at parity. If your threat model needs OS-level shell containment, run Shadow on macOS or Linux.

5 Operator responsibilities

- **Pick autonomy deliberately.** `auto-edit` is the shipped default; `full` is a trade you make.
- **Use a strong master password;** prefer the keychain over `SHADOW_VAULT_PASSWORD`.
- **Vet MCP servers** like a script you'd `curl | bash`.
- **On Windows,** add PowerShell denylist rules and don't assume sandbox behavior.
- **Strongest posture:** `--offline` with a local model — no cloud, no web tools, no MCP, no update check, sandbox egress denied.

Verification

`shadow doctor --privacy` — live egress report for your config, zero network calls · `shadow doctor` — sandbox/tool availability, so fail-open never surprises · **700+ tests** pin the security behaviors above on every release · an internal adversarial audit (14-agent red team, 66 verified findings — resolved or accepted and documented here) preceded the 2.5.x hardening line. Egress control today is enforced by construction and verified by test — not yet by an OS-level egress firewall; that distinction is why this section exists.

If you find a gap between this document and the code, that's a bug in one of them — please report it. Honesty here is a feature we ship. · ♦ Shadow — zero-telemetry, provider-neutral, local-first.